

ECS Fargate Nginx Server Deployment with ALB And EFS using Terraform (Task 5)



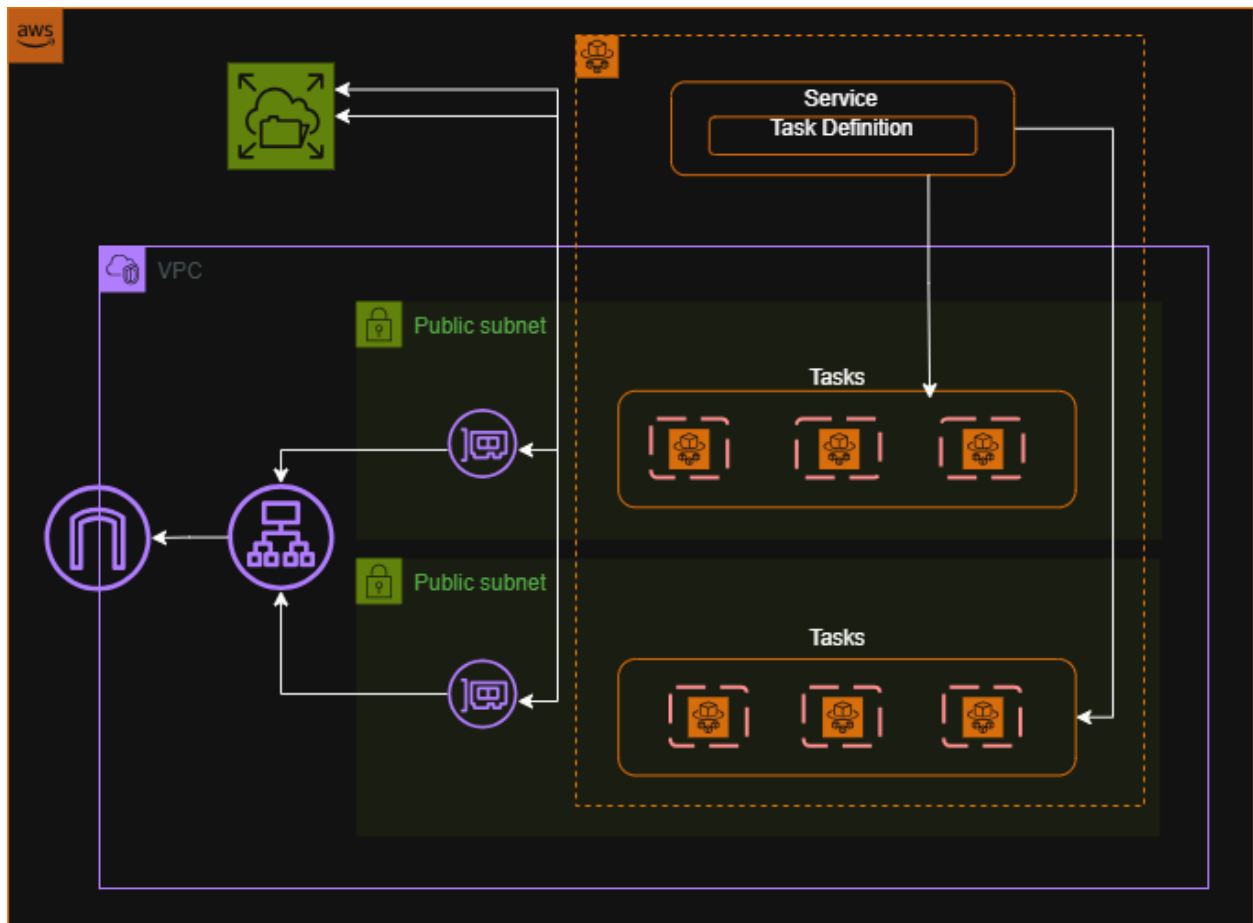
Zaeem Attique Ashar
Cloud Intern

Task Description:

This task will be a guide on setting up a Nginx server with a highly available architecture on AWS ECS Cluster Spread over multiple availability zones. An internet facing Load Balancer will be set up in front of the ECS cluster to listen to traffic and balance it upon tasks. AWS EFS will be used for persistence in shared storage to prevent data loss.

Architecture Diagram:	2
Task5.1: Design and create a highly available VPC with multiple public and private subnets	3
Task5.2: Configure security groups for ALB, ECS tasks, and EFS access	4
Task5.3: Create an ECS cluster with Terraform	5
Task5.4: Build and push a custom Docker image to a container registry	6
Task5.5: Define ECS task definition to use the custom Docker image and mount EFS volumes..	8
Task5.8: Set up and mount EFS for persistent storage in ECS tasks	11
Task5.9: Attach IAM roles and policies for ECS tasks and EFS access	12
Task5.10: Verify scalability and availability.....	13
Task5.11: Difficulties Faced	14

Architecture Diagram:



Task5.1: Design and create a highly available VPC with multiple public and private subnets

- VPC Configuration:
 - CIDR: 10.0.0.0/16.
- Subnets Configuration:

Availability Zone A:

- CIDR Block for subnet: 10.0.1.0/24.
- Availability Zone: us-west-2a.

Availability Zone B:

- CIDR Block for subnet: 10.0.2.0/24.
- Availability Zone: us-west-2a.

- Internet Gateway Configuration:
 - VPC ID.
- Route Table (Public):
 - VPC ID.
 - Route: Destination 0.0.0.0/0, Target Internet Gateway.
- Security Group (Public):
 - Inbound SSH Traffic: Port 22, source 0.0.0.0/0.
 - Inbound HTTP Traffic: Port 80, source 0.0.0.0/0.

```
1 resource "aws_vpc" "Task5-vpc-zaem" {
2   cidr_block = var.vpc_cidr
3   enable_dns_hostnames = true
4   enable_dns_support = true
5
6   tags = {
7     Name = "Task5-vpc-zaem"
8   }
9 }
10
11 resource "aws_subnet" "Task5-publicSNA-zaem" {
12   vpc_id = aws_vpc.Task5-vpc-zaem.id
13   cidr_block = var.subnet_cidr_A
14   availability_zone = "us-west-2a"
15   map_public_ip_on_launch = true
16
17   tags = {
18     Name = "Task5-publicSNA-zaem"
19   }
20 }
21
22 resource "aws_subnet" "Task5-publicSNB-zaem" {
23   vpc_id = aws_vpc.Task5-vpc-zaem.id
24   cidr_block = var.subnet_cidr_B
25   availability_zone = "us-west-2b"
26   map_public_ip_on_launch = true
27
28   tags = {
29     Name = "Task5-publicSNB-zaem"
30   }
31 }
32
33 resource "aws_internet_gateway" "Task5-igw-zaem" {
34   vpc_id = aws_vpc.Task5-vpc-zaem.id
35
36   tags = {
37     Name = "Task5-igw-zaem"
38   }
39 }
40
41 resource "aws_route_table" "Task5-publicRT-zaem" {
42   vpc_id = aws_vpc.Task5-vpc-zaem.id
43
44   route {
45     cidr_block = "0.0.0.0/0"
46     gateway_id = aws_internet_gateway.Task5-igw-zaem.id
47   }
48
49   tags = {
50     Name = "Task5-publicRT-zaem"
51   }
52 }
53
54 resource "aws_route_table_association"
55   "Task5-publicRTA-A-zaem" {
56     subnet_id = aws_subnet.Task5-publicSNA-zaem.id
57     route_table_id = aws_route_table.Task5-publicRT-zaem.id
58   }
59
60 resource "aws_route_table_association"
61   "Task5-publicRTA-B-zaem" {
62     subnet_id = aws_subnet.Task5-publicSNB-zaem.id
63     route_table_id = aws_route_table.Task5-publicRT-zaem.id
64   }
```

Task5.2: Configure security groups for ALB, ECS tasks, and EFS access

- Security Groups Configuration:
 - For Application Load Balancer:
 - Ingress port 80, source 0.0.0.0/0
 - Egress port any, destination 0.0.0.0/0
 - For Elastic Container Service task:
 - Ingress port 80, source ALB Security Group
 - Egress port any, destination 0.0.0.0/0
 - For Elastic File System
 - Ingress port 2049, source ECS Security Group
 - Egress port any, destination 0.0.0.0/0
- IAM Roles for Task Execution and Task:
 - Task Role Policies:
 - elasticfilesystem:ClientRootAccess
 - elasticfilesystem:ClientWrite
 - elasticfilesystem:ClientMount
 - Task Execution Role Policies:
 - AmazonECSTaskExecutionRolePolicy

```
1 resource "aws_security_group" "Task5-ALB-SG-Zaeem" {
2   name       = "Task5-ALB-SG-Zaeem"
3   vpc_id     = var.vpc_id
4
5   ingress {
6     from_port = 80
7     to_port   = 80
8     protocol  = "tcp"
9     cidr_blocks = [ "0.0.0.0/0" ]
10  }
11
12  egress {
13    from_port = 0
14    to_port   = 0
15    protocol  = "-1"
16    cidr_blocks = [ "0.0.0.0/0" ]
17  }
18
19  tags = {
20    Name = "Task5-ALB-SG-Zaeem"
21  }
22 }
23
24 resource "aws_security_group" "Task5-EFS-SG-Zaeem" {
25   name       = "Task5-EFS-SG-Zaeem"
26   vpc_id     = var.vpc_id
27
28   ingress {
29     from_port = 2049
30     to_port   = 2049
31     protocol  = "tcp"
32     security_groups = [ aws_security_group.Task5-ECS-SG-Zaeem.id ]
33   }
34
35   egress {
36     from_port = 0
37     to_port   = 0
38     protocol  = "-1"
39     cidr_blocks = [ "0.0.0.0/0" ]
40   }
41
42   tags = {
43     Name = "Task5-EFS-SG-Zaeem"
44   }
45 }
```

Task5.3: Create an ECS cluster with Terraform

- ECS Cluster Definition
 - Name: Task5-ECS-Cluster-Zaeem

- ECS Service Configuration:
 - Name: Task5-ECS-Service-Zaeem
 - Cluster ID passed
 - Task definition ARN passed
 - Desired Count: 2
 - Launch Type: Fargate
 - Load Balancer Configuration:
 - Target Group ARN passed
 - Container Name given.
 - Container port: Port that the container will listen on 80
 - Network Configuration:
 - Public Subnet A ID and Public Subnet B ID passed.
 - ECS Security Group ID passed.
 - Assign Public IP turned on.

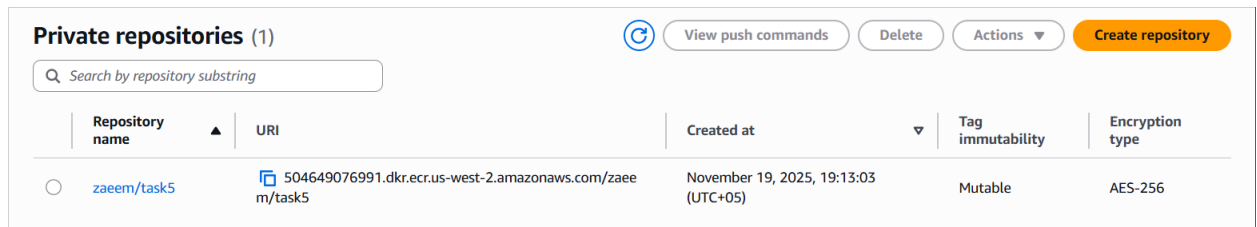
```
resource "aws_ecs_service" "Task5-ECS-Service-Zaeem" {
  name           = "Task5-ECS-Service-Zaeem"
  cluster        = aws_ecs_cluster.Task5-ECS-Cluster-Zaeem.id
  task_definition = aws_ecs_task_definition.Task5-TaskDefinition.arn
  desired_count  = var.task_desired_count
  launch_type    = "FARGATE"

  load_balancer {
    target_group_arn = var.tg_arn
    container_name   = var.container_name
    container_port   = 80
  }

  network_configuration {
    subnets          = [var.public_subnet_id_A, var.public_subnet_id_B]
    security_groups   = [var.ecs_sg_id]
    assign_public_ip  = true
  }
}
```

Task5.4: Build and push a custom Docker image to a container registry

1. Create a repository in the Elastic Container Registry (ECR).



2. Write the nginx server installation and configuration in a DockerFile in the root directory.

```
1 FROM nginx:alpine
2
3 # Remove default content
4 RUN rm -rf /usr/share/nginx/html/*
5
6 # Copy custom HTML files (if you have a directory with your website)
7 # COPY ./website /usr/share/nginx/html
8
9 # Create a simple custom page
10 RUN echo "<!DOCTYPE html>" > /usr/share/nginx/html/index.html && \
11     echo "<html>" >> /usr/share/nginx/html/index.html && \
12     echo "<head>" >> /usr/share/nginx/html/index.html && \
13     echo "    <title>Task5 - Zaeem</title>" >> /usr/share/nginx/html/index.html && \
14     echo "    <style>" >> /usr/share/nginx/html/index.html && \
15     echo "        body { font-family: Arial, sans-serif; text-align: center; padding: 50px; }" >> /usr/share/nginx/html/index.html && \
16     echo "        h1 { color: #333; }" >> /usr/share/nginx/html/index.html && \
17     echo "    </style>" >> /usr/share/nginx/html/index.html && \
18     echo "</head>" >> /usr/share/nginx/html/index.html && \
19     echo "<body>" >> /usr/share/nginx/html/index.html && \
20     echo "    <h1> Task5 - ECS with Nginx</h1>" >> /usr/share/nginx/html/index.html && \
21     echo "    <p>Successfully deployed on ECS Fargate</p>" >> /usr/share/nginx/html/index.html && \
22     echo "    <p>Created by: Zaeem</p>" >> /usr/share/nginx/html/index.html && \
23     echo "    <p>Container ID: ${HOSTNAME}</p>" >> /usr/share/nginx/html/index.html && \
24     echo "</body>" >> /usr/share/nginx/html/index.html && \
25     echo "</html>" >> /usr/share/nginx/html/index.html
26
27 # Copy custom nginx config (if needed)
28 # COPY nginx-custom.conf /etc/nginx/nginx.conf
29
30 # Expose port 80
31 EXPOSE 80
32
33 # Health check (optional)
34 HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
35     CMD curl -f http://localhost/ || exit 1
36
37 # Start Nginx in foreground
38 CMD ["nginx", "-g", "daemon off;"]
```

Explanation: Using the base image nginx:alpine which is a slim image of Nginx. Write the custom webpage in HTML and save it to /usr/share/nginx/html/index.html. Expose port 80 to listen to HTTP traffic. Use CMD directive to run the command 'nginx -g daemon off;' inside the container CLI to run the Nginx service and keep the task running.

3. Build the DockerFile into an Image

Command: `docker build -f <file name> -t <tag name> .`

```

PS C:\Users\zaeem\Documents\Innovation Lab - Cloudeelligent\Task5\Terraform> docker build -f DockerFile -t zaeem-nginx .
[+] Building 3.8s (8/8) FINISHED
=> [internal] load build definition from DockerFile                                0.0s
=> => transferring dockerfile: 1.83kB                                           0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine                  3.1s
=> [auth] library/nginx:pull token for registry-1.docker.io                    0.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.0s
=> [1/3] FROM docker.io/library/nginx:alpine@sha256:b3c656d55d7ad751196f21b7fd2e8d4da9cb430e32f646adc92441b72f8 0.1s
=> => resolve docker.io/library/nginx:alpine@sha256:b3c656d55d7ad751196f21b7fd2e8d4da9cb430e32f646adc92441b72f8 0.0s
=> CACHED [2/3] RUN rm -rf /usr/share/nginx/html/*                             0.0s
=> CACHED [3/3] RUN echo "<!DOCTYPE html>" > /usr/share/nginx/html/index.html && echo "<html>" >> /usr/share 0.0s
=> exporting to image                                                            0.3s
=> => exporting layers                                                            0.0s
=> => exporting manifest sha256:33ff5269e707a6fa75cbcd503bda9c48dadf8b8bf5305feb3c2edfd7f1942852      0.0s
=> => exporting config sha256:6fd8eab547b1d00611c7202f740690999445e8fc8a557ab35c4bf3a30bfff6686      0.0s
=> => exporting attestation manifest sha256:f78b0dcd804fd08bec6410025775418e90620af68b1c1d076b8c2d595e97af0d 0.1s
=> => exporting manifest list sha256:ccb5ad421d1bfc4bbd158e03283a506f3bd228774b42895920ea1d7c6c6581e2 0.0s
=> => naming to docker.io/library/zaeem-nginx:latest                           0.0s
=> => unpacking to docker.io/library/zaeem-nginx:latest                         0.0s

```

4. Push the image to the ECR:

Command: *docker push <ECR Repository URI>*

Task5.5: Define ECS task definition to use the custom Docker image and mount EFS volumes

- ECS Task Definition
 - Family: Task5-Zaeem
 - Task Role ARN: ARN of the IAM Role created for the task.
 - Execution Role ARN: ARN of the IAM Role created for execution of task.
 - Cluster Mode: FARGATE
 - Network Mode: awsvpc
 - CPU: 256
 - Memory: 512
 - Container definition:
 - Image: 504649076991.dkr.ecr.us-west-2.amazonaws.com/zaeem/task5
 - Port Mappings: Container port 80 -> Host port 80
 - Mount points: Source volume Task5-EFS-Zaeem, container path /mnt/data
 - Volume Configuration: EFS ID passed, Root dir "/", encryption enabled, port 2999, Access Point ID passed

```
resource "aws_ecs_task_definition" "Task5-TaskDefinition" {
  depends_on = [ var.efs_id ]
  family     = "Task5-Zaeem"
  task_role_arn = var.Task5-ecs-task-role-arn
  execution_role_arn = var.Task5-ecs-task-execution-role-arn
  requires_compatibilities = ["FARGATE"]
  network_mode = "awsvpc"
  cpu          = "256"
  memory       = "512"
  container_definitions = jsonencode([
    {
      name       = "NginxServer"
      image      = "504649076991.dkr.ecr.us-west-2.amazonaws.com/zaeem/task5:latest"
      # cpu      = 256
      # memory   = 512
      essential = true
      portMappings = [
        {
          containerPort = 80
          hostPort      = 80
        }
      ]
      mountPoints = [
        {
          sourceVolume = "Task5-EFS-Zaeem"
          containerPath = "/mnt/data"
        }
      ]
    }
  ])
}

volume {
  name = "Task5-EFS-Zaeem"

  efs_volume_configuration {
    file_system_id = var.efs_id
    root_directory = "/"
    transit_encryption = "ENABLED"
    transit_encryption_port = 2999
    authorization_config {
      access_point_id = var.efs_ap_id
      iam             = "ENABLED"
    }
  }
}
```

Task5.6: Create an Application Load Balancer (ALB) with proper listeners and target groups

- Load Balancer Configuration:
 - Name: Task5-ALB-Zaeem
 - Internal: False (The ALB is internet facing)
 - Load Balancer Type: Application
 - Security Groups: ALB Security Group ID attached
 - Subnets: Public Subnet A ID and Public Subnet B ID passed
 - Enable Deletion Protection: false (To allow terraform to manage the ELB)
- Target Group Configuration:
 - Name: Task5-TG-Zaeem
 - Port: 80
 - Protocol: HTTP
 - Target Type: IP (ECS will attach the Task IP)
 - VPC ID: Task 5 VPC ID passed
- Load Balancer Listener:
 - Load Balancer ARN passed
 - Port to listen on: 80
 - Protocol to listen for: HTTP
 - Default Action: Forward traffic to Target Group ARN

```
1  resource "aws_lb" "Task5-ALB-Zaeem" {
2      name           = "Task5-ALB-Zaeem"
3      internal       = false
4      load_balancer_type = var.load_balancer_type
5      security_groups = [var.alb_sg_id]
6      subnets       = [var.public_subnet_id_A, var.public_subnet_id_B]
7
8      enable_deletion_protection = false
9  }
10
11
12  resource "aws_lb_target_group" "Task5-TG-Zaeem" {
13      name           = "Task5-TG-Zaeem"
14      port           = 80
15      protocol       = "HTTP"
16      target_type    = var.tg_target_type
17      vpc_id         = var.vpc_id
18  }
19
20  resource "aws_lb_listener" "front_end" {
21      load_balancer_arn = aws_lb.Task5-ALB-Zaeem.arn
22      port              = var.lb_listener_port
23      protocol          = var.lb_listener_protocol
24
25      default_action {
26          type = "forward"
27          target_group_arn = aws_lb_target_group.Task5-TG-Zaeem.arn
28      }
29  }
```

Task5.7: Configure ECS service to use Fargate with ALB integration for load balancing

- ECS Service Configuration:
 - Name: Task5-ECS-Service-Zaeem
 - Cluster ID passed
 - Task definition ARN passed
 - Desired Count: 2
 - Launch Type: Fargate
 - Load Balancer Configuration:
 - Target Group ARN passed
 - Container Name given.
 - Container port: Port that the container will listen on 80

```
resource "aws_ecs_service" "Task5-ECS-Service-Zaeem" {
  name           = "Task5-ECS-Service-Zaeem"
  cluster        = aws_ecs_cluster.Task5-ECS-Cluster-Zaeem.id
  task_definition = aws_ecs_task_definition.Task5-TaskDefinition.arn
  desired_count  = var.task_desired_count
  launch_type    = "FARGATE"

  load_balancer {
    target_group_arn = var.tg_arn
    container_name   = var.container_name
    container_port   = 80
  }

  network_configuration {
    subnets          = [var.public_subnet_id_A, var.public_subnet_id_B]
    security_groups   = [var.ecs_sg_id]
    assign_public_ip  = true
  }
}
```

Task5.8: Set up and mount EFS for persistent storage in ECS tasks

- Add the local mount point for the EFS volume in the container configuration under the mountPoints block:
 - Source Volume: Name of the EFS Volume
 - Container Path: /mnt/data
- Configure the Volume block for the task:
 - Name: Name the volume
 - Under the efs_volume_configuration:
 - File system ID: EFS ID passed
 - Root Directory: "/" Default
 - Encryption: Enabled
 - Encryption Port: 2999
 - Access Point ID passed.

```
resource "aws_ecs_task_definition" "Task5-TaskDefinition" {
  depends_on = [ var.efs_id ]
  family = "Task5-Zaeem"
  task_role_arn = var.Task5-ecs-task-role-arn
  execution_role_arn = var.Task5-ecs-task-execution-role-arn
  requires_compatibilities = ["FARGATE"]
  network_mode = "awsvpc"
  cpu = "256"
  memory = "512"
  container_definitions = jsonencode([
    {
      name = "NginxServer"
      image = "504649076991.dkr.ecr.us-west-2.amazonaws.com/zaeem/task5:latest"
      # cpu = 256
      # memory = 512
      essential = true
      portMappings = [
        {
          containerPort = 80
          hostPort = 80
        }
      ]
      mountPoints = [
        {
          sourceVolume = "Task5-EFS-Zaeem"
          containerPath = "/mnt/data"
        }
      ]
    }
  ])

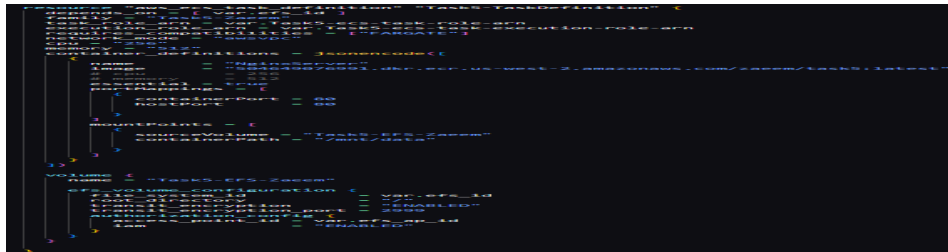
  volume {
    name = "Task5-EFS-Zaeem"

    efs_volume_configuration {
      file_system_id = var.efs_id
      root_directory = "/"
      transit_encryption = "ENABLED"
      transit_encryption_port = 2999
      authorization_config {
        access_point_id = var.efs_ap_id
        iam = "ENABLED"
      }
    }
  }
}
```

Task5.9: Attach IAM roles and policies for ECS tasks and EFS access

Under ECS Task Definition, we need to attach the roles for Task Execution and the Task running.

- Task Role ARN passed
- Execution Role ARN passed



Task5.10: Verify scalability and availability

To verify availability, delete one of the running tasks. In response, the service running should detect that the cluster is not at the desired state of 2 running tasks and should automatically provision another task from the task definition.

Task5-ECS-Service-Zaeem [Info](#) Last updated November 19, 2025, 22:11 (UTC+5:00) [Delete service](#) [Update service](#)

Service overview [Info](#)

Status: Active | Tasks (2 Desired): 0 Pending | 2 Running | Task definition: revision [Task5-Zaeem:14](#) | Deployment status: Success

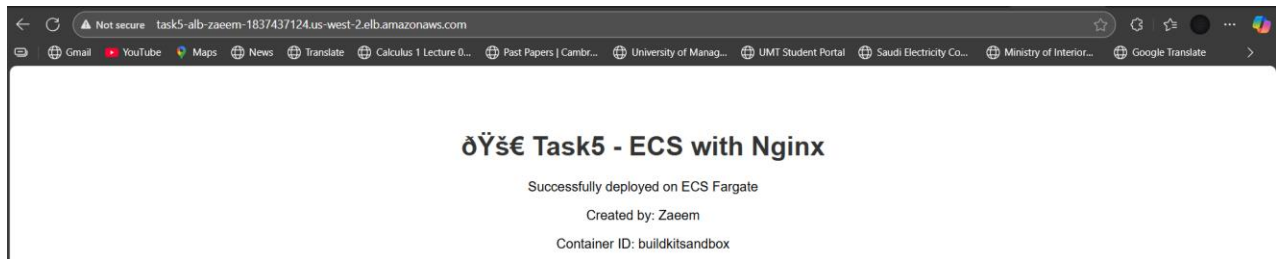
Health and metrics | **Tasks** | Logs | Deployments | Events | Configuration and networking | Service auto scaling | Event history | Tags

Tasks (1/3) Last updated November 19, 2025, 22:12 (UTC+5:00) [Stop](#)

Filter tasks by property or value | Filter desired status: Any desired status | Filter launch type: Any launch type

Task	Last status	Desired st...	T...	Health sta...	Created at	Started by	Started at	Conta
becd2e57689a4d408ff4b...	Running	Running	Tas...	Unknown	3 hours ago	ecs-svc/41836278803...	3 hours ago	-
e1b6c11f99a944978d5be...	Provisioning	Running	Tas...	Unknown	11 seconds ago	ecs-svc/41836278803...	-	-
ae8350e2487246a3a4c1...	Deactivating	Stopped	Tas...	Unknown	3 hours ago	ecs-svc/41836278803...	3 hours ago	-

To access the home page of the Nginx Server, enter the DNS name into the browser.



Task5.11: Difficulties Faced

- The ECS task by default tries to connect to the EFS Mount Points in the subnet using the DNS name, and it throws an error when it is unable to resolve it. When creating a custom VPC, we need to manually enable the DNS and Hostname resolution feature for the ECS Task to do that.